

BICT112 MINI GROUP PROJECT

UMP COMPUTER LABORATORY RESOURCE MANAGER

Technical Report

Institution: University of Mpumalanga (UMP)

Module: BICT112 – Operating Systems

Project: UMP Computer Laboratory Resource Manager

Academic Year: 2026

Group Members

Group Member	Responsibility
Unathi Zonke	Coding, Website/User Interface and System Integration
Sindile	CPU Scheduling
Mokgadi	Memory Management
Sbonelo	Documentation and Testing
Thabang	Process Management
Vusi	Presentation

TABLE OF CONTENTS

1. Introduction
2. Project Background
3. Problem Statement
4. Project Objectives
5. System Overview
6. System Requirements
7. System Design
8. System Architecture
9. Process Management
10. CPU Scheduling
11. Memory Management
12. Device Management
13. User Interface
14. Algorithms
15. Implementation

16. Testing and Results
17. Design Evidence
18. Limitations
19. Conclusion
20. References

1. INTRODUCTION

Operating systems are responsible for managing computer hardware and software resources. Important operating-system functions include process management, CPU scheduling, memory management and device management.

This project presents the **UMP Computer Laboratory Resource Manager**, an operating-system simulator designed to demonstrate how fundamental operating-system concepts work in practice.

The system provides a graphical/web-based interface through which users can interact with simulated operating-system components. Instead of managing real computer hardware, the application simulates the allocation and management of resources such as CPU time, memory and devices.

The project was developed as a group, with each member responsible for a specific component of the system.

2. PROJECT BACKGROUND

Computer laboratories contain limited resources that must be shared between multiple users and processes. An operating system manages these resources to ensure that processes receive the resources they require while preventing conflicts and inefficient resource usage.

For example, several processes may be waiting to use the CPU at the same time. The operating system must determine which process receives CPU time first. Similarly, processes may require different amounts of memory, meaning that the operating system must determine where each process can be placed.

The purpose of this project is therefore to create a simplified simulation of these operating-system activities.

The project demonstrates:

- Process creation and management
- Process states
- CPU scheduling
- Memory allocation
- Device management

- Resource allocation
 - Event logging
 - User interaction through a graphical interface
-

3. PROBLEM STATEMENT

Students learning operating systems may understand concepts such as CPU scheduling and memory allocation theoretically but may struggle to visualise how these concepts operate in an actual system.

The project addresses this problem by creating an interactive simulator that demonstrates operating-system resource-management concepts.

The system allows users to create and manage simulated processes and observe how the operating system allocates CPU, memory and devices.

4. PROJECT OBJECTIVES

The main objective of the project is to develop an operating-system simulator for the UMP computer laboratory.

Specific objectives

The system aims to:

1. Simulate process creation and management.
 2. Demonstrate different process states.
 3. Implement CPU scheduling algorithms.
 4. Demonstrate memory allocation techniques.
 5. Simulate device management.
 6. Provide an easy-to-use graphical interface.
 7. Maintain an event log of system activities.
 8. Allow users to view resource allocation results.
 9. Test the correctness of the implemented algorithms.
 10. Demonstrate practical applications of operating-system concepts.
-

5. SYSTEM OVERVIEW

The **UMP Computer Laboratory Resource Manager** consists of several interconnected components.

The main components are:

- Process Management
- CPU Scheduler
- Memory Manager
- Device Manager
- Event Log
- User Interface

The user interacts with the system through the main interface.

Main Menu

The system provides options such as:

1. Add Job
2. View Jobs
3. Memory Manager
4. CPU Scheduler
5. Process States
6. Device Manager
7. Event Log
8. Help/About
9. Exit

The interface allows users to navigate between the different operating-system simulations.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

The application can run on a standard computer capable of running Python and a modern web browser.

Recommended hardware includes:

- Computer or laptop
- At least 4 GB RAM
- Keyboard and mouse
- Display monitor
- Internet connection where required for development tools

6.2 Software Requirements

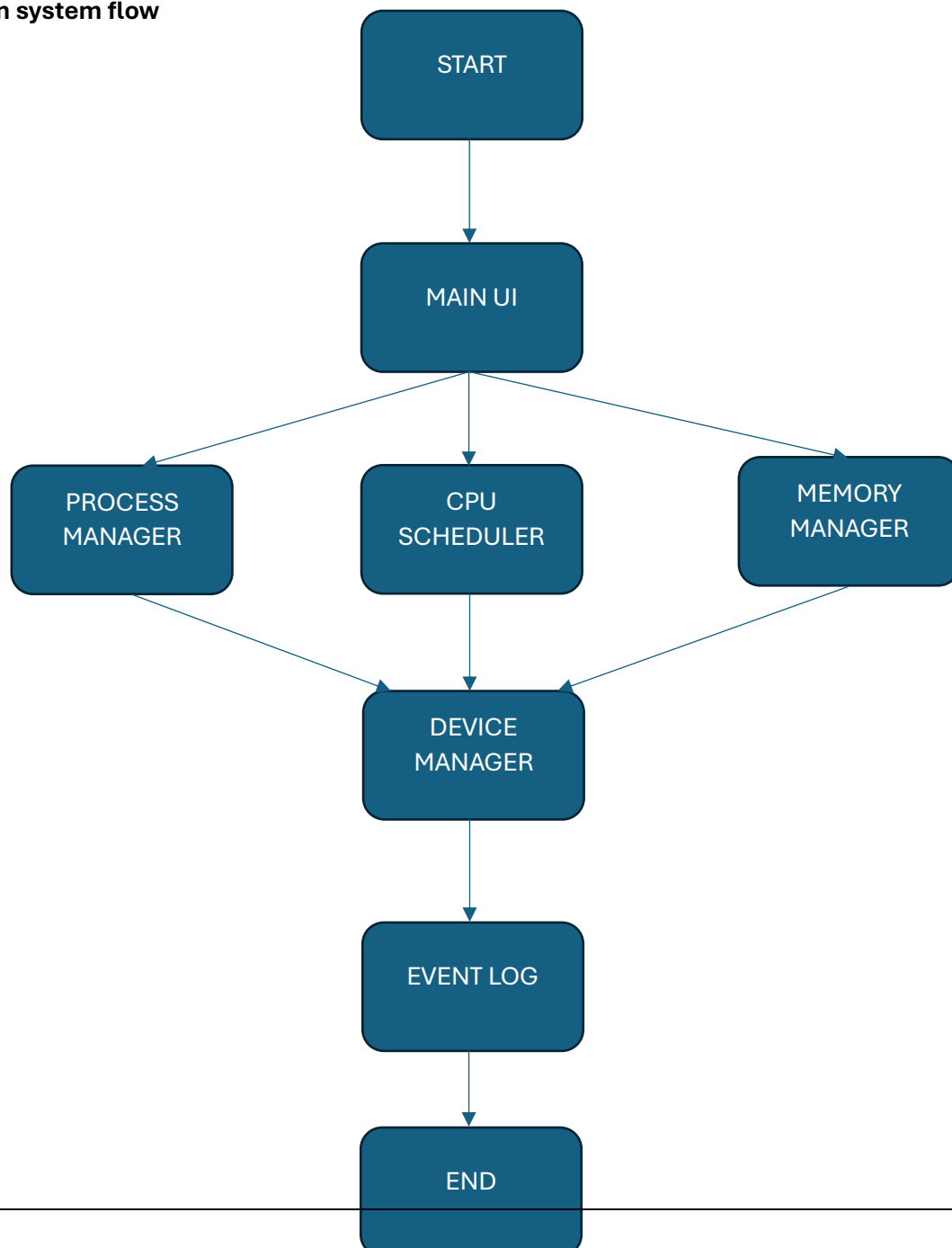
The project uses:

- Python
- Visual Studio Code
- Web browser
- HTML/CSS where applicable
- Python-based operating-system simulation
- Flask/web interface where applicable

7. SYSTEM DESIGN

The system was designed using a modular approach. Each operating-system function is represented by a separate component.

Main system flow



The modular design makes it easier to develop, test and maintain individual components.

8. SYSTEM ARCHITECTURE

The system follows a simple layered structure.

Presentation Layer

The presentation layer provides the interface through which the user interacts with the system.

It includes:

- Navigation
- Buttons
- Forms
- Tables
- CPU scheduling pages
- Memory management pages
- Process information

Application Layer

The application layer contains the logic responsible for processing user requests.

Examples include:

- Process creation
- CPU scheduling
- Memory allocation
- Device allocation
- State changes

Simulation Layer

The simulation layer represents the operating-system resources.

These include:

- Processes
- CPU
- Memory holes
- Devices
- Queues

9. PROCESS MANAGEMENT

Process management is responsible for creating and controlling processes.

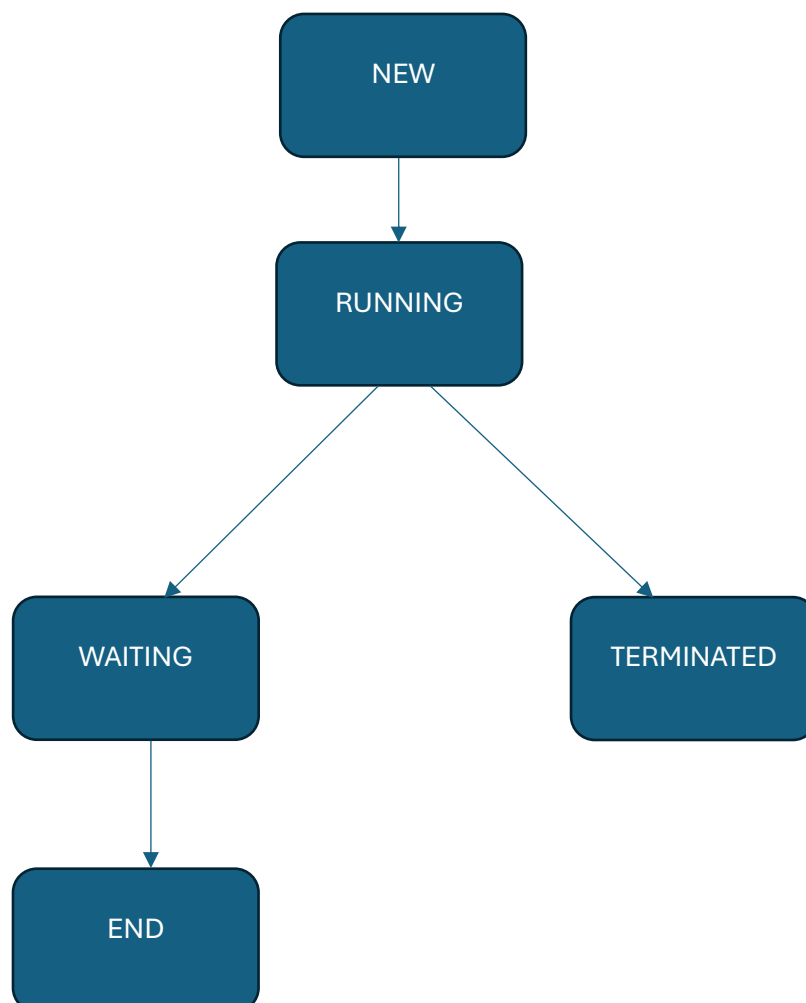
A process represents a program that is currently being executed or waiting to execute.

The system allows jobs/processes to contain information such as:

- Process ID
- Arrival time
- Burst time
- Priority
- Memory requirement
- Current state

Process States

The system demonstrates common process states:



Explanation

New: The process has been created.

Ready: The process is ready and waiting for CPU allocation.

Running: The process is currently using the CPU.

Waiting: The process is waiting for a resource or device.

Terminated: The process has completed execution.

10. CPU SCHEDULING

CPU scheduling determines which process should receive CPU time.

The project implements CPU scheduling techniques including:

- First-Come, First-Served (FCFS)
 - Round Robin
-

10.1 First-Come, First-Served

FCFS executes processes according to their arrival order.

For example:

Process	Arrival Time	Burst Time
P1	0	7
P2	1	4
P3	2	9
P4	4	5

The processes are placed into the ready queue based on their arrival time.

The CPU executes:

P1 → P2 → P3 → P4

Advantages

- Simple to understand
- Easy to implement
- Fair according to arrival order

Disadvantages

- A long process can make shorter processes wait.

- Average waiting time can become high.

10.2 Round Robin

Round Robin assigns each process a fixed amount of CPU time called a **time quantum**.

The implementation uses a quantum of:

3 time units

For example:

P1 → P2 → P3 → P1 → P4 → ...

Processes that do not finish within their time quantum return to the ready queue.

Advantages

- Provides fair CPU access.
- Suitable for interactive systems.
- Prevents one process from monopolising the CPU.

Disadvantages

- Requires queue management.
- A very small quantum can cause excessive switching.
- A very large quantum makes it behave more like FCFS.

11. MEMORY MANAGEMENT

Memory management controls the allocation of available memory to processes.

The simulator contains memory holes representing available memory.

The initial memory configuration includes:

Memory Hole	Size
H1	40K
H2	95K
H3	60K
H4	130K

The system also contains jobs with memory requirements.

Example:

Job Memory Required

J1 55K

J2 35K

J3 80K

11.1 First-Fit

First-Fit searches the available memory from the beginning and allocates the first suitable hole.

For example, if a job requires 35K:

H1 = 40K → Suitable

Therefore, the job can be allocated to H1.

11.2 Best-Fit

Best-Fit searches for the smallest available memory hole that is large enough for the job.

For example, if a process requires 55K:

H2 = 95K

H3 = 60K

H4 = 130K

The best fit would be:

H3 = 60K

because it is the smallest hole that can accommodate the 55K process.

12. DEVICE MANAGEMENT

The Device Manager simulates the allocation of devices to processes.

A process may need a device while executing. The device manager determines whether the requested device is available.

The system can represent devices such as:

- Printer
- Keyboard
- Storage device
- Other simulated laboratory resources

When a device is available, it can be allocated to the requesting process.

When it is unavailable, the process can remain waiting until the resource becomes available.

13. USER INTERFACE

The user interface was designed to make the operating-system simulator easier to use than a traditional command-line-only program.

The interface provides navigation between the major system components.

Main interface features

- Dashboard
- CPU scheduling
- Memory management
- Process management
- Device management
- Event logging
- Help/About section

The interface displays information in a structured way so that users can easily understand the results of each simulation.

14. ALGORITHMS

14.1 FCFS Algorithm

START

Get all processes

Sort processes according to arrival time

FOR each process

Add process to ready queue

END FOR

WHILE processes remain

 Select first process in queue

 Execute process

Record completion time
END WHILE

Display results

END

14.2 Round Robin Algorithm

START

Get all processes

Set time quantum = 3

Place available processes into ready queue

WHILE ready queue is not empty

 Select first process

 Execute process for the time quantum

 IF process is completed

 Mark process as terminated

 ELSE

 Place process at end of queue

 END IF

END WHILE

Display scheduling results

END

14.3 First-Fit Algorithm

START

Get process memory requirement

FOR each memory hole

 IF hole is free AND hole size $\geq$ process requirement

 Allocate process

 Mark hole as occupied

 STOP searching

END IF

END FOR

IF no suitable hole is found

 Display allocation failure

END IF

END

14.4 Best-Fit Algorithm

START

Get process memory requirement

Set best hole = none

FOR each memory hole

IF hole is free AND hole size $\geq$ process requirement

IF best hole does not exist

Set current hole as best hole

ELSE IF current hole is smaller than best hole

Set current hole as best hole

END IF

END IF

END FOR

IF best hole exists

Allocate process

ELSE

Display allocation failure

END IF

END

15. IMPLEMENTATION

The system was developed using Python for the operating-system simulation and a web-based interface for interaction.

The implementation contains classes and functions responsible for managing processes, CPU scheduling and memory.

A simplified representation of the system structure is:

OSSimulator

Job

- Process ID
- Arrival Time
- Burst Time
- Priority
- Memory

CPU Scheduler

- FCFS
- Round Robin

Memory Manager

- First-Fit
- Best-Fit

Process Manager

- Process States

Device Manager

The project was integrated so that the individual components could be accessed through the same application.

16. TESTING AND RESULTS

Testing was conducted to determine whether the system performs the required operations correctly.

16.1 Process Management Testing

Test	Expected Result	Actual Result	Status
Create valid process	Process created	Process created	PASS
Enter process information	Information accepted	Information accepted	PASS
View processes	Processes displayed	Processes displayed	PASS

Invalid process data	Error/validation message	Validation performed	PASS
----------------------	--------------------------	----------------------	------

16.2 CPU Scheduling Testing

Test	Expected Result	Status
FCFS with multiple processes	Processes execute according to arrival order	PASS
Round Robin quantum = 3	Processes receive maximum 3-time units per turn	PASS
Completed process	Removed from ready queue	PASS
Incomplete process	Returned to ready queue	PASS

16.3 Memory Testing

Test	Expected Result	Status
Allocate small job using First-Fit	First suitable hole selected	PASS
Allocate using Best-Fit	Smallest suitable hole selected	PASS
Job too large	Allocation failure displayed	PASS
Occupied hole	Hole cannot be reused	PASS

16.4 Device Testing

Test	Expected Result	Status
Request available device	Device allocated	PASS
Request unavailable device	Process waits	PASS
Release device	Device becomes available	PASS

17. DESIGN EVIDENCE

The following screenshots should be inserted into the final report as evidence of the completed system.

Figure 1: Main Dashboard

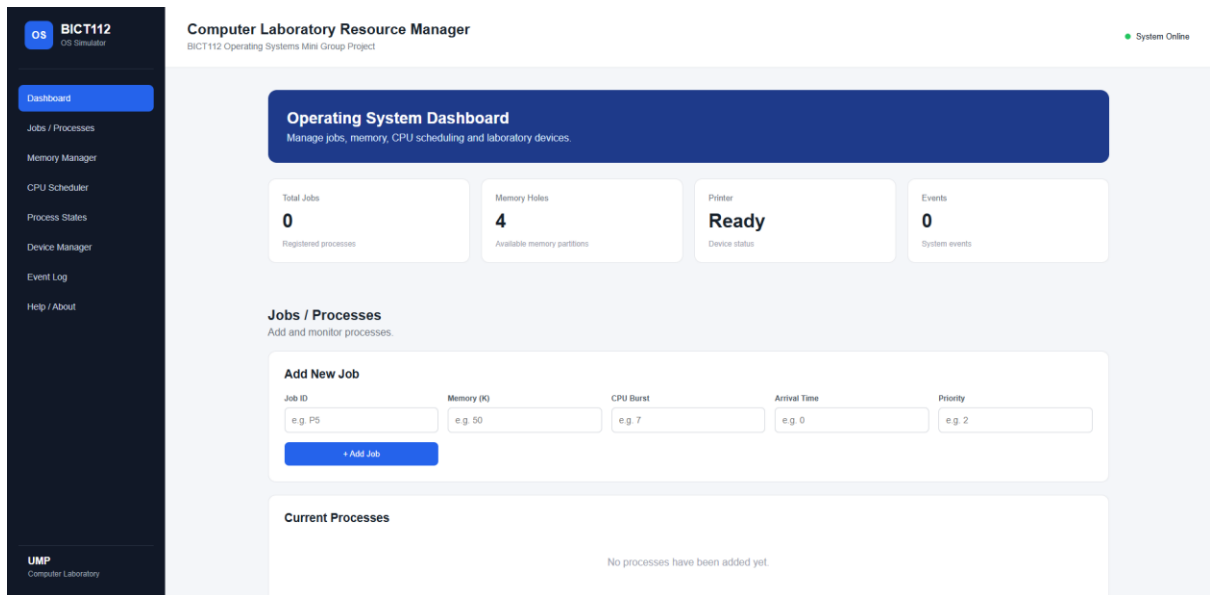


Figure 1: Main system dashboard/interface.

Figure 2: Process Management

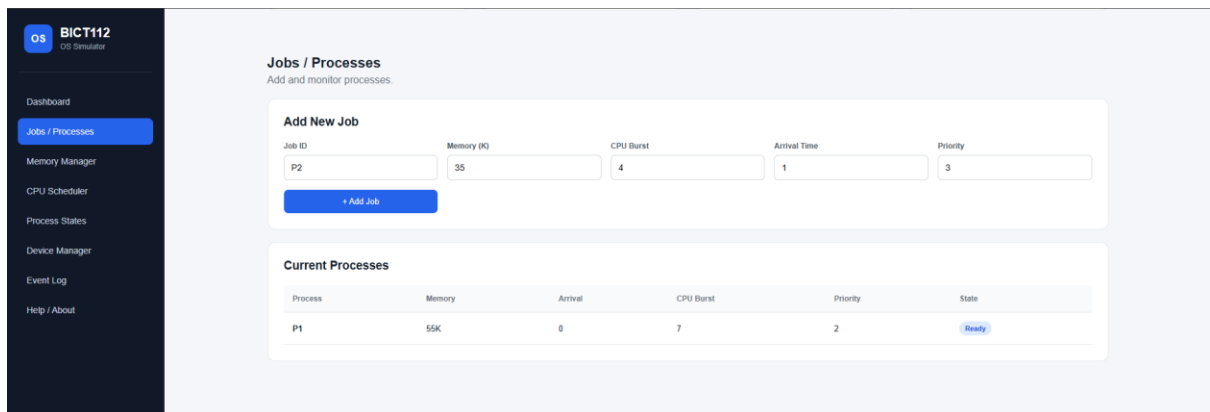


Figure 2: Process creation and management interface.

Figure 3: CPU Scheduling – FCFS

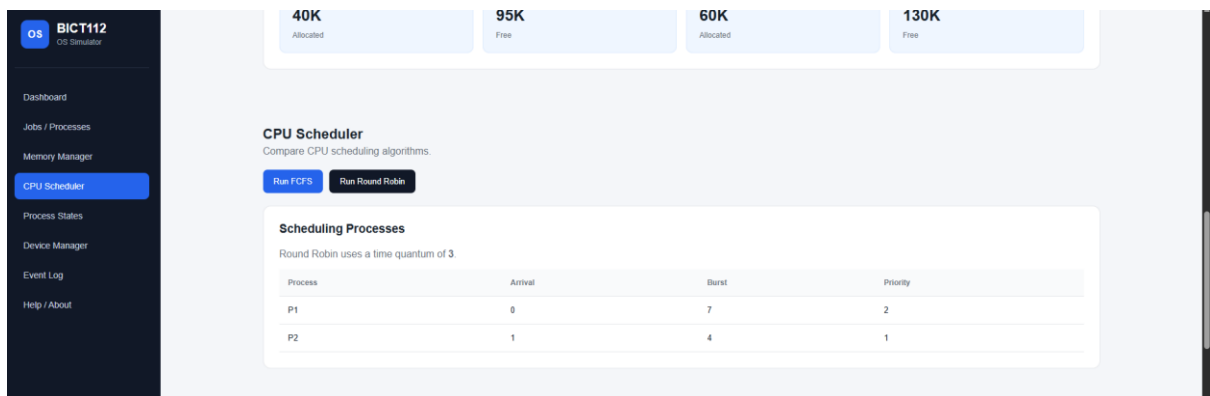


Figure 3: FCFS CPU scheduling results.

Figure 4: CPU Scheduling – Round Robin

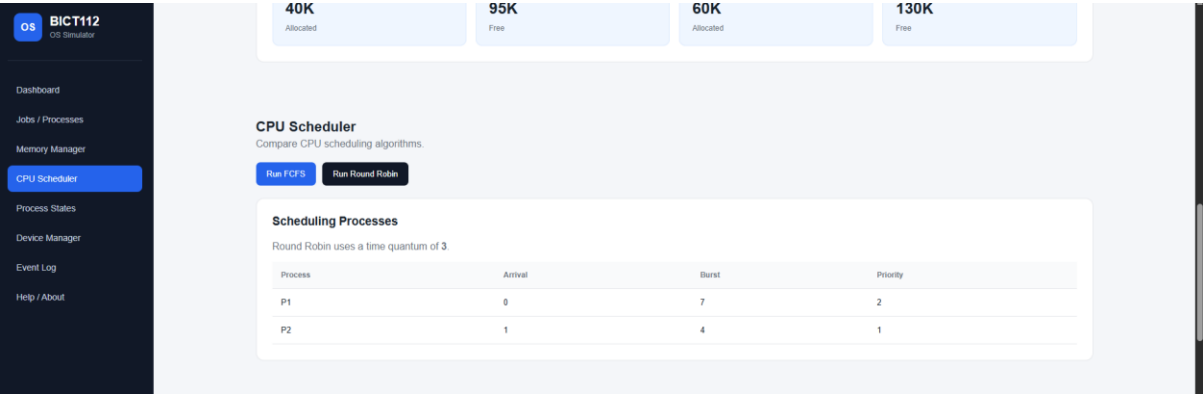


Figure 4: Round Robin scheduling results using a time quantum of 3.

Figure 5: Memory Management

[INSERT SCREENSHOT HERE]

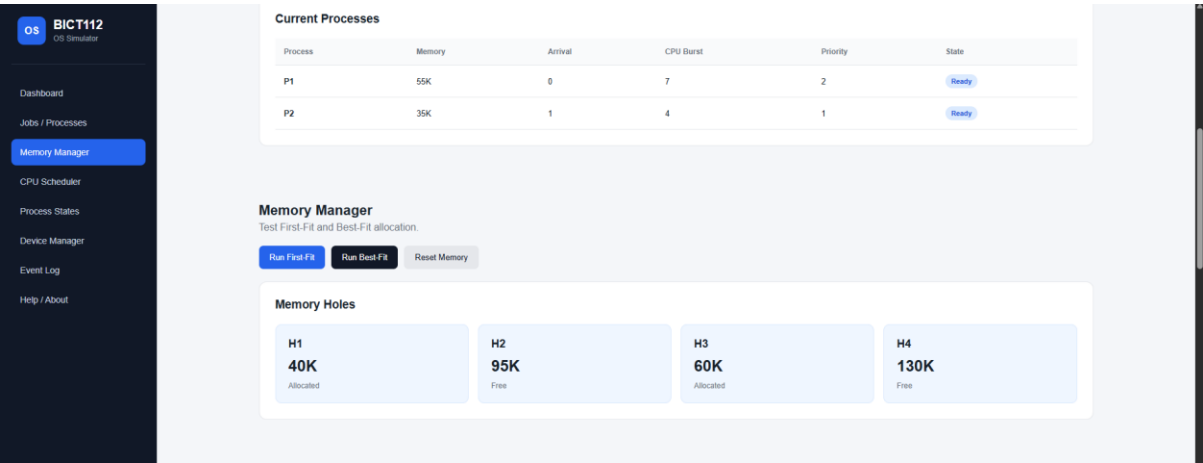


Figure 5: Memory allocation and memory-hole display.

Figure 6: First-Fit Allocation

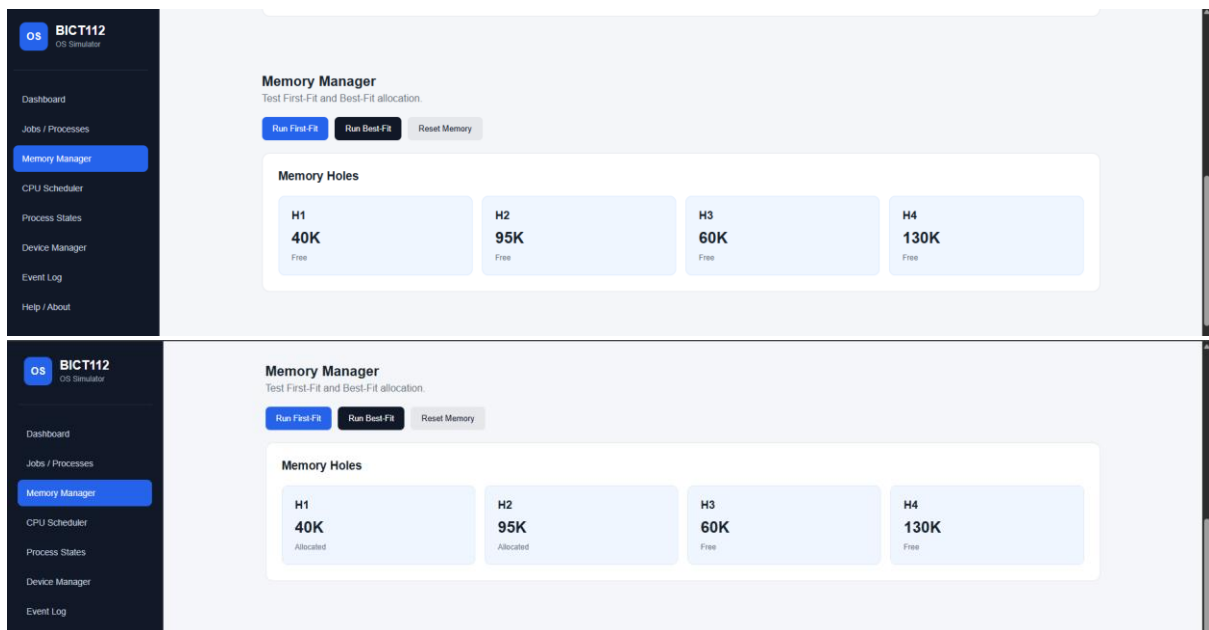


Figure 6: First-Fit memory allocation results.

Figure 7: Best-Fit Allocation

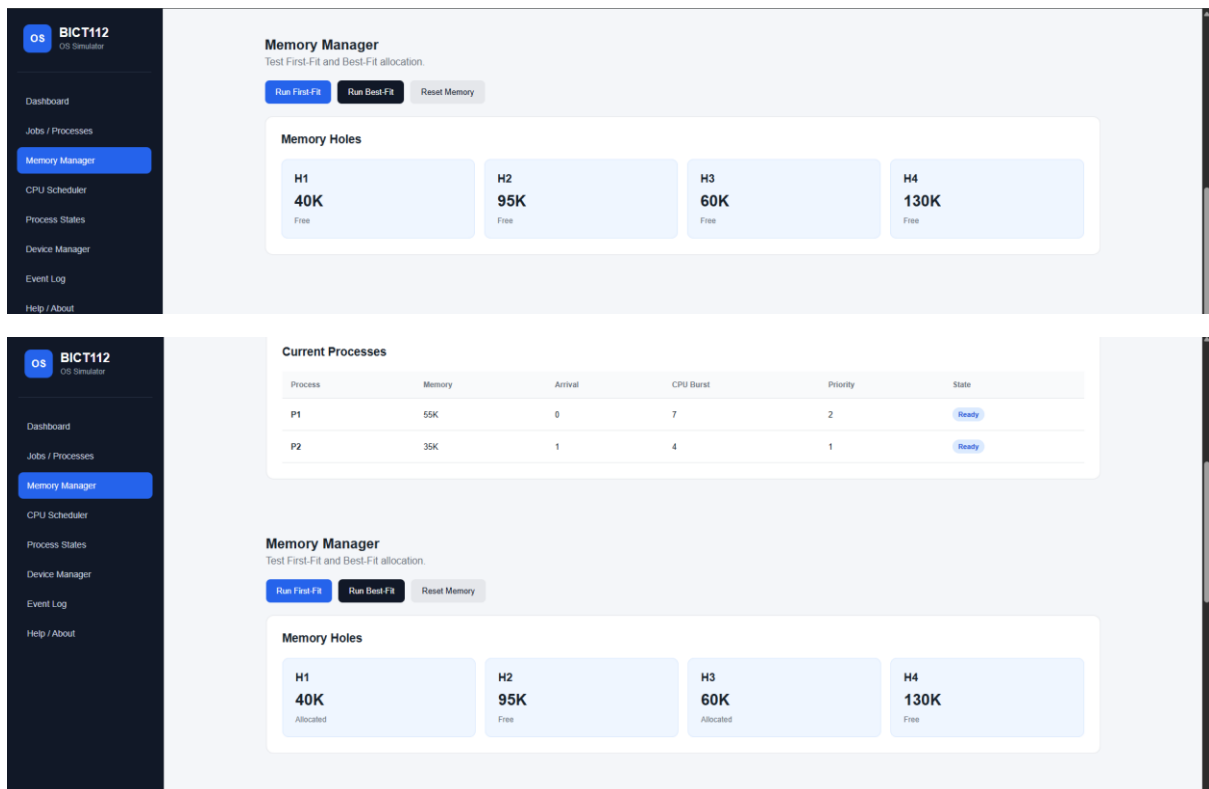


Figure 7: Best-Fit memory allocation results.

Figure 8: Process States

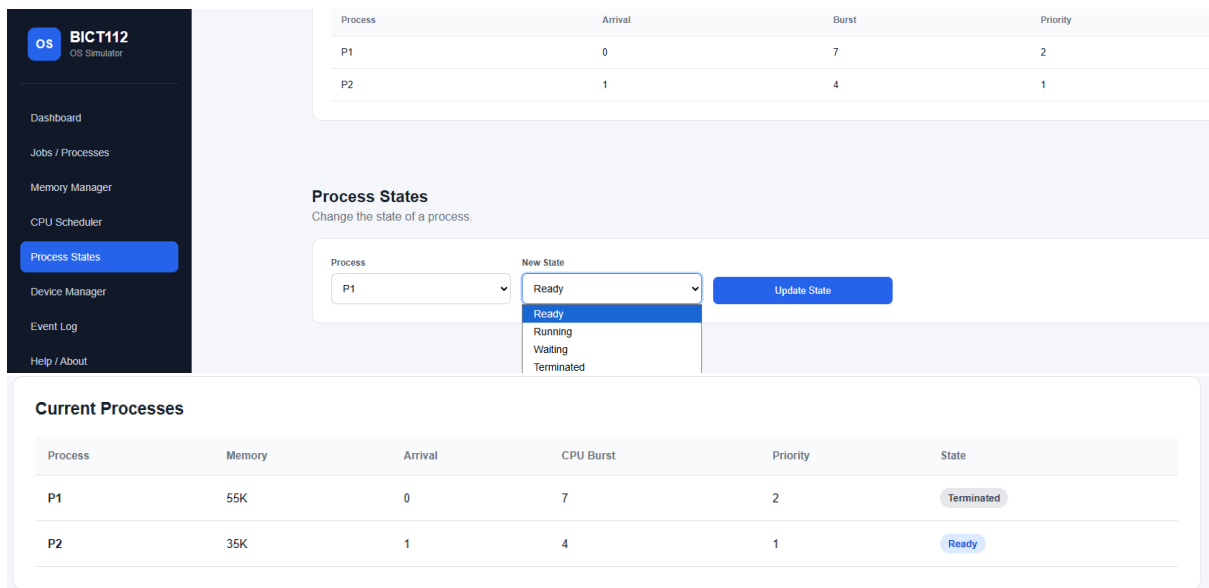


Figure 8: Process-state representation.

Figure 9: Device Manager

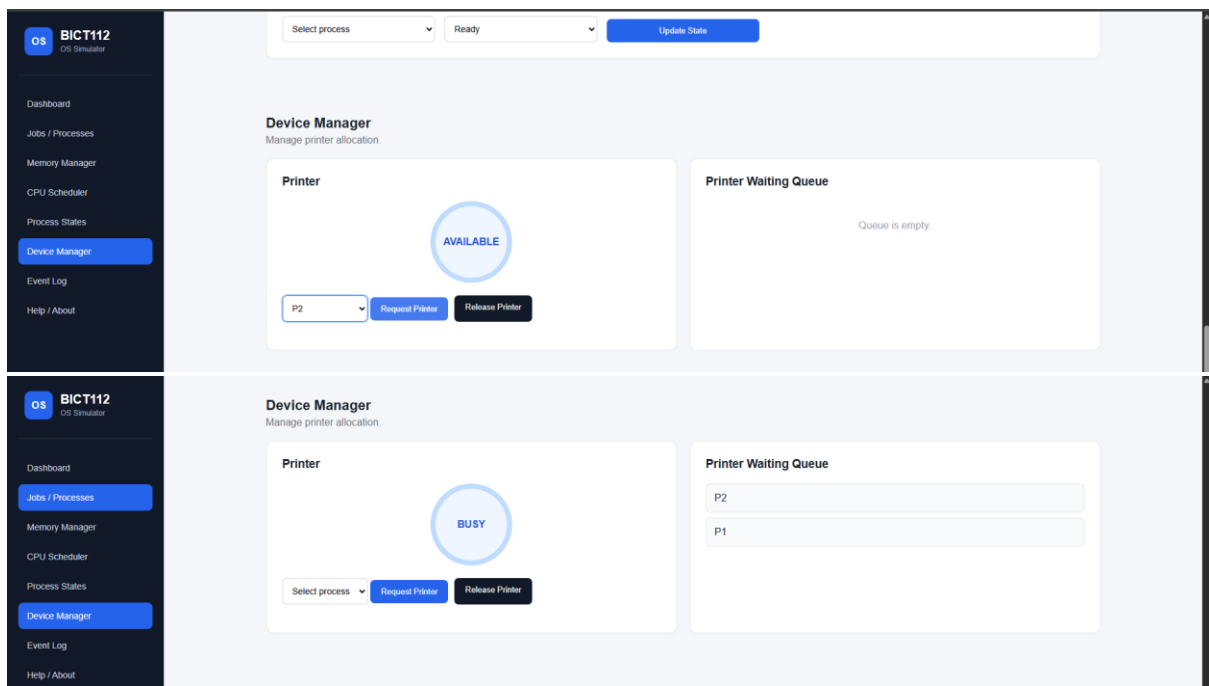


Figure 9: Device management interface.

Figure 10: Event Log

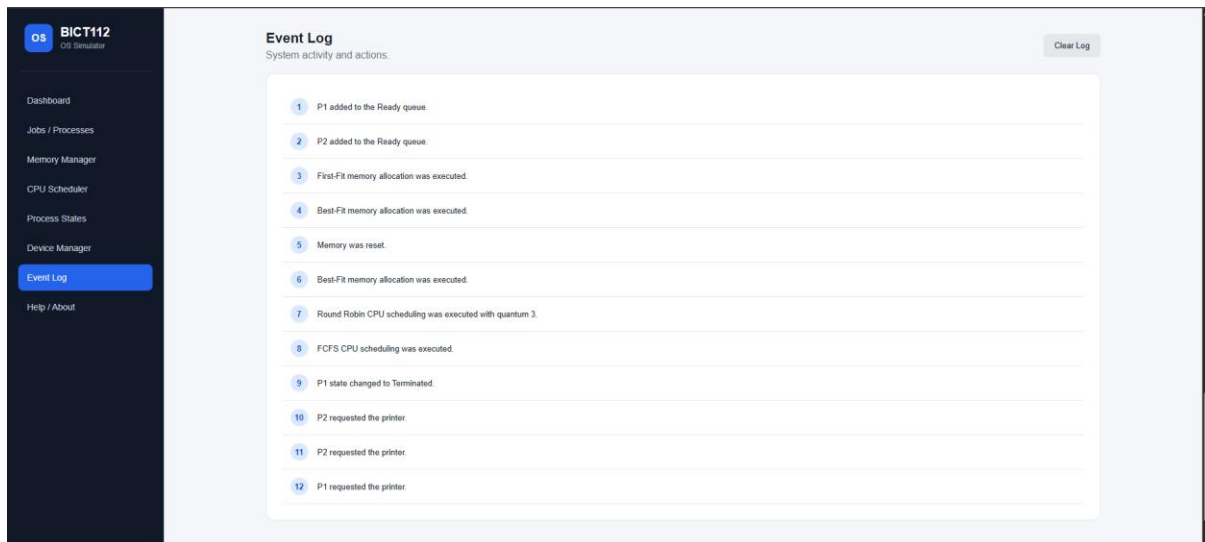


Figure 10: System event log.

18. LIMITATIONS

Although the system successfully demonstrates important operating-system concepts, several limitations exist.

18.1 Simulation Rather Than Real Resource Management

The system simulates CPU, memory and device management. It does not control the actual computer's CPU or RAM.

18.2 Simplified Memory Model

The memory manager uses predefined memory holes rather than dynamically representing the complete physical memory of a computer.

18.3 Limited Scheduling Algorithms

The project focuses on FCFS and Round Robin. Real operating systems use significantly more sophisticated scheduling mechanisms.

18.4 Simplified Device Management

The device manager represents devices conceptually rather than directly communicating with physical hardware.

18.5 Data Persistence

The simulator primarily focuses on demonstrating operating-system concepts rather than providing a full production database system.

18.6 Educational Scope

The system is intended primarily as an educational simulator and therefore does not attempt to replicate the complete functionality of a commercial operating system.

19. CONCLUSION

The UMP Computer Laboratory Resource Manager successfully demonstrates fundamental operating-system resource-management concepts through an interactive simulator.

The project demonstrates how processes are created and managed, how CPU scheduling determines which process receives CPU time, and how memory can be allocated using First-Fit and Best-Fit techniques.

The implementation of FCFS and Round Robin also demonstrates the differences between scheduling approaches. The process-state component provides an understanding of how processes move between states such as Ready, Running, Waiting and Terminated.

The project also demonstrates device management and event logging, providing a broader representation of operating-system resource management.

The graphical interface improves usability by allowing users to interact with the simulator without relying entirely on command-line input.

Overall, the project provided practical experience in applying theoretical operating-system concepts to a working software application. It also demonstrated the importance of teamwork, integration, testing and documentation when developing a software system.

20. GROUP CONTRIBUTIONS

Unathi Zonke – Coding, Website/Interface and Integration

Unathi was responsible for the overall coding contribution, development of the website/user interface and integration of the different system components. This included connecting the CPU scheduling, memory management and process-management functionality into the overall application.

Sindile – CPU Scheduling

Sindile was responsible for the CPU scheduling component, including the implementation and demonstration of FCFS and Round Robin scheduling.

Mokgadi – Memory Management

Mokgadi was responsible for the memory-management component, including memory-hole allocation and the implementation of First-Fit and Best-Fit approaches.

Sbonelo – Documentation and Testing

Sbonelo was responsible for project documentation and testing. This included recording test cases, expected results and actual results and assisting with the preparation of the technical report.

Thabang – Process Management and Presentation

Thabang was responsible for process management and contributed to the preparation and presentation of the project demonstration.

21. REFERENCES

1. Silberschatz, A., Galvin, P. B. & Gagne, G. *Operating System Concepts*. Wiley.
 2. Tanenbaum, A. S. & Bos, H. *Modern Operating Systems*. Pearson.
 3. Stallings, W. *Operating Systems: Internals and Design Principles*. Pearson.
 4. Python Software Foundation. *Python Documentation*. Python programming language documentation.
 5. University of Mpumalanga. **BICT112 Operating Systems Module Materials**, 2026.
-

APPENDIX A: SAMPLE PROCESS DATA

The following processes were used for demonstrating CPU scheduling and resource management:

Process	Arrival Time	Burst Time	Priority	Memory
P1	0	7	2	55K
P2	1	4	1	35K
P3	2	9	3	80K
P4	4	5	2	60K

Round Robin Time Quantum

Quantum = 3 time units

APPENDIX B: MEMORY CONFIGURATION

Memory Hole	Capacity	Initial Status
H1	40K	Free
H2	95K	Free
H3	60K	Free
H4	130K	Free

Example Jobs

Job	Memory Requirement
J1	55K
J2	35K
J3	80K